

Отчет по итогам прохождения курса по предмету «Кроссплатформенное программное обеспечение»

Введение

В рамках данного курса была изучена современная экосистема разработки серверных приложений на языке Java с использованием фреймворка Spring Boot. Итоговым практическим заданием стала разработка системы «Smart Home» (Умный дом). Данная система представляет собой REST API, позволяющее управлять устройствами, мониторить показатели датчиков и настраивать автоматизацию домашних процессов.

Актуальность работы заключается в растущем интересе к интернету вещей (IoT) и необходимости создания безопасных, масштабируемых систем для управления бытовой электроникой. В ходе проекта были реализованы ключевые требования современной разработки: безопасность, логирование, документация и интеграция с внешними сервисами.

Что было изучено в ходе курса

В процессе обучения и реализации проекта были освоены следующие ключевые технологии и архитектурные подходы:

- Основы Spring Boot 3 и управление зависимостями: Изучена структура проекта, работа с файлом `pom.xml` и автоматическое управление библиотеками через Maven.
- Работа с системой контроля версий Git: Изучили основы командной разработки: создание репозитория, фиксацию изменений (commits) и управление историей проекта. Использование Git позволило отслеживать прогресс и обеспечило сохранность кода на различных этапах разработки.
- Работа с базами данных (JPA/Hibernate): Реализовано взаимодействие с СУБД PostgreSQL. Мы научились описывать сущности (Devices, Sensors, Users) и использовать репозитории для выполнения CRUD-операций (создание, чтение, обновление, удаление).
- Безопасность (Spring Security и JWT): Одной из самых сложных тем стала настройка безопасности. Была реализована авторизация на основе JSON Web Tokens (JWT), где токен хранится в защищенных куках (HttpOnly). Это позволяет системе «узнавать» пользователя без необходимости повторного ввода пароля при каждом запросе.
- Ролевая модель и разрешения (Permissions): Внедрена система разделения прав. С помощью аннотации `@PreAuthorize` над методами контроллеров (например, `device:read` или `automationrule:manage`) удалось гибко настроить доступ: обычный пользователь может только смотреть состояние сенсоров, а администратор — изменять их значение.
- Документирование API (Swagger): Для удобства использования API другими разработчиками была настроена библиотека Swagger. Она автоматически создает страницу в браузере, где описаны все доступные методы (эндпоинты) системы.
- Интеграция с внешними сервисами (Telegram API): Интегрировали отправление уведомлений в телеграм бот. Это позволяет оперативно получать информацию о

регистрации новых пользователей или критических событиях в системе прямо на телефон.

- Работа с файлами и отчетами: Были реализованы функции экспорта конфигурации системы в формат JSON и импорта данных из файлов. Это необходимо для резервного копирования настроек «Умного дома».
- Логирование и мониторинг: С помощью библиотеки Slf4j и собственного LogService реализовано полное логирование действий пользователей. Информация о каждом входе или изменении устройства сохраняется в базе данных и выводится в консоль.
- Формирование отчетности (iText/JasperReports): Были изучены библиотеки для программного создания документов. Научились динамически формировать PDF-файлы на основе данных из базы, настраивать шрифты, таблицы и стили в документах.

Реализация проекта

В качестве темы для финального проекта была выбрана система «Умный дом». Это решение позволяет автоматизировать бытовые процессы и обеспечивать удаленный контроль над жильем.

Структура базы данных. В основе системы лежит реляционная база данных. Логическая схема включает в себя следующие ключевые таблицы:

- users и user_roles — для хранения данных пользователей и их прав доступа.
- devices — реестр всех подключенных умных устройств.
- sensors — хранение данных с датчиков (температура, влажность и т.д.).
- rooms — организация устройств по помещениям.
- automation_rules — правила, по которым система выполняет действия автоматически.
- activity_logs — история всех действий в системе.

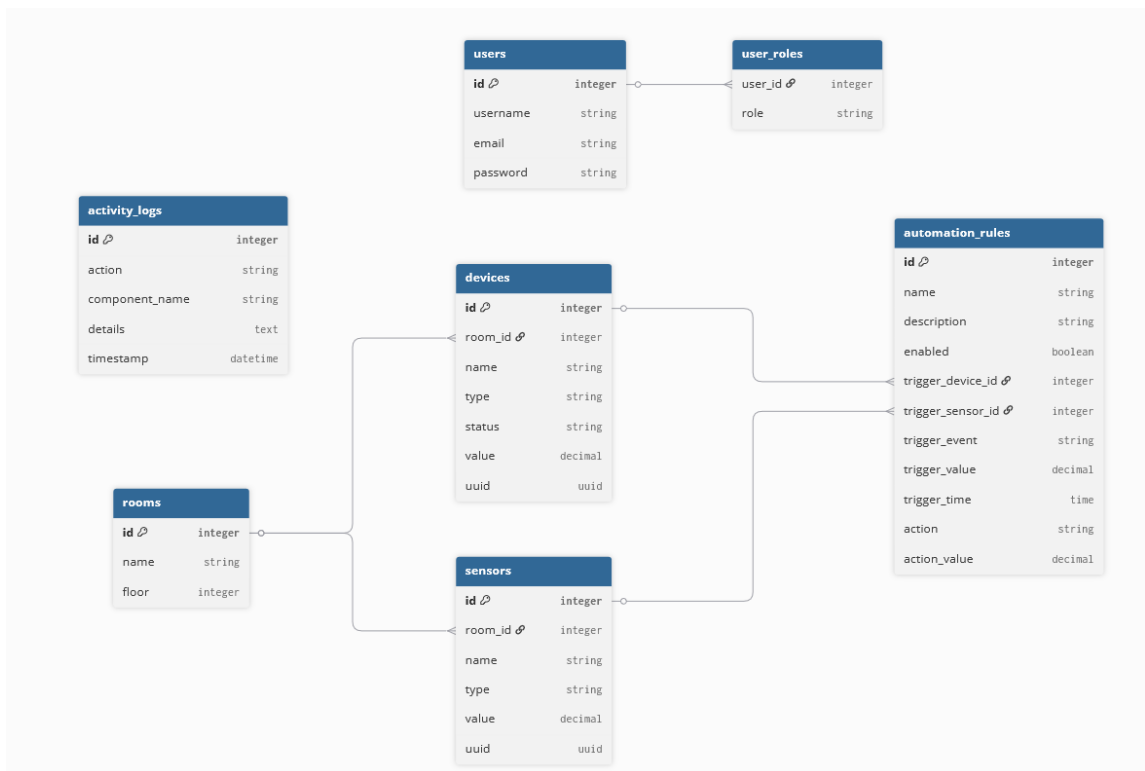


Рисунок 1 - Логическая схема базы данных

Функциональные возможности. В приложении реализован полный цикл управления данными (CRUD). Пользователи могут создавать, просматривать, изменять и удалять устройства и правила.

- **Авторизация и роли:** Доступ к системе строго разграничен. Реализована ролевая модель с использованием Permissions. Каждому методу в коде предшествует аннотация `@PreAuthorize`, которая проверяет, есть ли у пользователя право на конкретное действие (например, «удаление устройства» доступно только администратору)
- **Интеграция с Telegram:** Система настроена на отправку уведомлений. При возникновении важных событий (например, регистрация нового пользователя или срабатывание датчика) бот мгновенно отправляет сообщение в чат владельцу.
- **Работа с файлами и отчетами:** Одной из ключевых функций является формирование PDF-отчета. Система анализирует данные из таблицы логов и датчиков, после чего формирует документ со сводной статистикой. Также реализована возможность загрузки и чтения конфигурационных файлов.

SMART HOME FULL SYSTEM REPORT

1. SYSTEM STATUS REPORT

Current state of rooms, devices and sensors

Room: Гостиная (Floor 1)

Entity Name	Type	Status/Value	UUID (Last 4)
Умная лампа (Device)	LIGHT	OFF	...b72a
Датчик движения (Sensor)	MOTION	31.0	...87a1

Рисунок 2 - Секция отчета 1. Вывод комнат, их девайсов и сенсоров, с типом, статусом, значением(если есть) и UUID

2. SYSTEM AUTOMATION RULES

Правила автоматизации

Rule Name	Trigger Condition	Action	State
Включить свет при движении	If Датчик движения GREATER_THAN 30.0	TURN_ON on Умная лампа	ACTIVE
Включить свет по времени	At 05:02	TURN_OFF on Умная лампа	ACTIVE

Рисунок 3 - Секция отчета 2. Вывод правил автоматизации, их условия для выполнения, действия, и состояние

3. SYSTEM ACTIVITY LOG

Недавние события

Timestamp	Component	Action	Details
2026-01-26T05:01:07.061904	AutomationRule	CREATED	{ "id": 2, "name": "Включить свет по времени", "description": "Это правило выключает свет по времени.", "enabled": true, "triggerDevice": { "id": 1, "name": "Умная лампа", "type": "LIGHT", "status": "ON", "roomId": 1, "uuid": "edd52397-85b5-4fcc-bf2d-13f60285b72a", "value": null }, "triggerSensor": null, "triggerEvent": "TIME", "triggerValue": null, "triggerTime": "05:02:00", "action": "TURN_OFF", "actionValue": null }
2026-01-26T04:59:42.066157	Sensor	UPDATED	{ "id": 1, "name": "Датчик движения", "type": "MOTION", "value": 31.0, "roomId": 1, "uuid": "ca684c0f-be4a-4016-88ba-f344b92d87a1" }

Рисунок 4 - Секция отчета 3. Вывод истории событий, время, действие, компонент (к которому относится действие) и детали (в основном в json формате)

Перспективы развития системы

В будущем систему можно расширить следующими модулями:

1. Интеграция с реальным оборудованием (IoT): Подключение протоколов MQTT для управления реальными контроллерами (ESP32, Raspberry Pi) в режиме реального времени.
2. Модуль аналитики и графиков: Внедрение фронтенд-части для визуализации данных с датчиков в виде динамических графиков (например, изменение температуры за неделю).
3. Сложные сценарии: Добавление поддержки сложных условий автоматизации (например, «если датчик движения сработал И время после 23:00 — включить ночное освещение»).

Применение полученных знаний в будущем

Знания, полученные на курсе, имеют прямое практическое применение:

1. Backend-разработка: Знание Spring Boot востребовано в большинстве крупных компаний и банков, так как это основной инструмент для создания надежных серверных решений.
2. Проектирование систем: Умение составлять схемы БД и распределять права доступа (Permissions) пригодится при создании любого сервиса с личными кабинетами.
3. Автоматизация: Принципы работы с внешними API (Telegram) и генерация документов (PDF) могут быть использованы в проектах по автоматизации бизнес-процессов.
4. Культура разработки: Регулярное использование Git приучает к порядку в коде и упрощает работу в команде.

Заключение

В ходе выполнения итогового проекта была успешно разработана комплексная система «Умный дом», построенная на стеке Java Spring Boot. Процесс реализации позволил не только закрепить теоретические основы объектно-ориентированного программирования, но и получить глубокое практическое понимание того, как функционируют современные многоуровневые веб-приложения. Были решены ключевые задачи: проектирование реляционной базы данных, внедрение системы безопасности с использованием JWT-токенов, настройка гибкой ролевой модели и автоматизация бизнес-логики через систему правил и уведомлений. Главный результат — это не просто работающая программа, а освоение профессионального стека технологий, который станет фундаментом для дальнейшего роста в качестве разработчика.